

CSC103-Programming Fundamentals

Lecture-6

Instructor

Rizwan Rashid

Lecture Outline

- Increment/Decrement Operators
- Formatted Output

Increment and Decrement Operators

The increment operator (++) and decrement operator (--) are for incrementing and decrementing a variable by 1.

Example – PostIncrement / Postdecrement

```
int i = 3, j = 3;  
i++; // i becomes 4  
j--; // j becomes 2
```

Example – PreIncrement / Predecrement

```
int i = 3, j = 3;  
++i; // i becomes 4  
--j; // j becomes 2
```

Increment and Decrement Operators

```
int i = 10;
```

```
int newNum = 10 * i++;
```

Same effect as

```
int newNum = 10 * i;  
i = i + 1;
```

```
int i = 10;
```

```
int newNum = 10 * (++i);
```

Same effect as

```
i = i + 1;  
int newNum = 10 * i;
```

Increment and Decrement Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>	<i>Example (assume i = 1)</i>
++var	preincrement	Increment var by 1 , and use the new var value in the statement	int j = ++i; // j is 2, i is 2
var++	postincrement	Increment var by 1 , but use the original var value in the statement	int j = i++; // j is 1, i is 2
--var	predecrement	Decrement var by 1 , and use the new var value in the statement	int j = --i; // j is 0, i is 0
var--	postdecrement	Decrement var by 1 , and use the original var value in the statement	int j = i--; // j is 1, i is 0

Character Data Type

- Computers use binary numbers internally
- Mapping a character to its binary representation is called *encoding*
- How characters are encoded is defined by an *encoding scheme*
- Java supports **Unicode** (16 bit character encoding and 65,536 characters are possible)
- A 16-bit Unicode takes **two bytes**
 - Preceded by `\u`, expressed in four hexadecimal digits
 - i.e. ranges from `\u0000` To `\uFFFF`

ASCII Code for Commonly Used Characters

- Unicode includes ASCII code with \u0000 to \u007F corresponding to the 128 ASCII characters

Characters	Code Value in Decimal	Unicode Value
'0' to '9'	48 to 57	\u0030 to \u0039
'A' to 'Z'	65 to 90	\u0041 to \u005A
'a' to 'z'	97 to 122	\u0061 to \u007A

Character Data Type

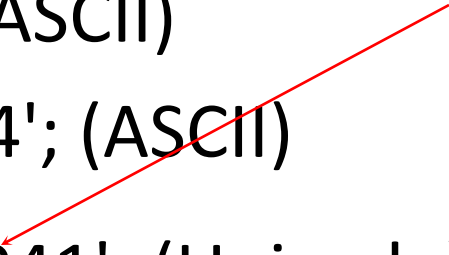
`char letter = 'A'; (ASCII)`

`char numChar = '4'; (ASCII)`

`char letter = '\u0041'; (Unicode)`

`char numChar = '\u0034'; (Unicode)`

Four hexadecimal digits.



NOTE: The increment and decrement operators can also be used on char variables to get the next or preceding Unicode character. For example, the following statements display character b.

```
char ch = 'a';
```

```
System.out.println(++ch);
```

Escape Sequences for Special Characters

	Escape Sequence	Description
<code>\n</code>	Newline	Cursor moves to the beginning of the next line
<code>\t</code>	Tab	Cursor moves to the next tab stop
<code>\b</code>	Backspace	Cursor moves one space to the left
<code>\r</code>	Return	Cursor moves to the beginning of the current line (not the next line)
<code>\\</code>	Backslash	Backslash is printed
<code>\'</code>	Single quotation	Single quotation mark is printed
<code>\"</code>	Double quotation	Double quotation mark is printed

Casting between char and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';
```

```
char c = 97; // Same as char c = (char)97;
```

```
char ch = (char)65.25; // Decimal 65 is assigned to ch
```

```
System.out.println(ch); // ch is character A
```

- All numeric operators can be applied to char operands

```
int i = '2' + '3'; // (int)'2' is 50 and (int)'3' is 51
```

```
System.out.println("i is " + i); // i is 101
```

```
int j = 2 + 'a'; // (int)'a' is 97
```

```
System.out.println("j is " + j); // j is 99
```

```
System.out.println(j + " is the Unicode for character "
```

```
    + (char)j); // 99 is the Unicode for character c
```

```
System.out.println("Chapter " + '2');
```

i is 101

j is 99

99 is the Unicode for character c

Chapter 2

Formatting Console Output

You can use the `System.out.printf` method to display formatted output on the console.

- **Syntax**

```
System.out.printf(format, item1, item2,..., itemk);
```

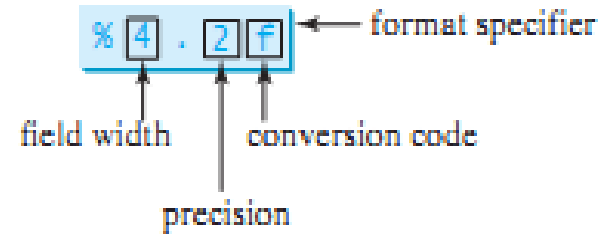
- Where `format` is a string that may consist of substrings and format specifier.
- A format specifier specifies how an item should be displayed.
- An item may be a numeric value, character, boolean value, or a string.
 - Items must match the format specifier in order, in number, and in exact type
- A simple format specifier consists of a percent sign (%) followed by a conversion code e.g. (`%d`).

Frequently-Used Specifiers

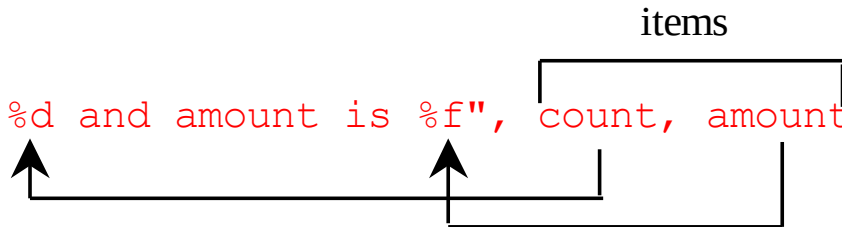
Specifier	Output	Example
<code>%b</code>	a boolean value	true or false
<code>%c</code>	a character	'a'
<code>%d</code>	a decimal integer	200
<code>%f</code>	a floating-point number	45.460000
<code>%e</code>	a number in standard scientific notation	4.556000e+01
<code>%s</code>	a string	"Java is cool"

Formatting Console Output

```
double amount = 12618.98;  
double interestRate = 0.0013;  
double interest = amount * interestRate;  
System.out.printf("Interest is $%4.2f",  
    interest);
```



```
int count = 5;  
double amount = 45.56;  
System.out.printf("count is %d and amount is %f", count, amount);
```



display count is 5 and amount is 45.560000

Width and Precision in Format Specifier

<i>Example</i>	<i>Output</i>
%5c	Output the character and add four spaces before the character item, because the width is 5.
%6b	Output the Boolean value and add one space before the false value and two spaces before the true value.
%5d	Output the integer item with width at least 5. If the number of digits in the item is < 5 , add spaces before the number. If the number of digits in the item is > 5 , the width is automatically increased.
%10.2f	Output the floating-point item with width at least 10 including a decimal point and two digits after the point. Thus, there are 7 digits allocated before the decimal point. If the number of digits before the decimal point in the item is < 7 , add spaces before the number. If the number of digits before the decimal point in the item is > 7 , the width is automatically increased.
%10.2e	Output the floating-point item with width at least 10 including a decimal point, two digits after the point and the exponent part. If the displayed number in scientific notation has width less than 10, add spaces before the number.
%12s	Output the string with width at least 12 characters. If the string item has fewer than 12 characters, add spaces before the string. If the string item has more than 12 characters, the width is automatically increased.

Example

```
System.out.printf("%8d%8s%8.1f\n", 1234, "Java", 5.63);  
System.out.printf("%-8d%-8s%-8.1f \n", 1234, "Java", 5.63);
```

display

```
|← 8 →|← 8 →|← 8 →|  
□□□□ 1234 □□□□ Java □□□□ 5.6  
1234 □□□□ Java □□□□ 5.6 □□□□
```

where the square box (□) denotes a blank space.